UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 9 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Source: CSC429-2019-06, page [2]

Unofficial model answer:

a) Two main components of the Von Neumann Architecture: (1) the Central Processing Unit (CPU), comprising the Control Unit and the Arithmetic Logic Unit (ALU), and (2) the Main Memory (store), which holds both program instructions and data. Other acceptable components include the input/output (I/O) devices and the buses connecting them; the defining pair is the CPU and the single shared memory.

b) A general-purpose computer is a machine that is not hard-wired for one fixed task; it can be programmed to perform many different kinds of computation by loading a different sequence of instructions (a program) into its memory. Because instructions and data share the same store and are fetched and executed by the same hardware, the same physical machine can solve a wide variety of problems simply by changing its program, rather than by rewiring or rebuilding the hardware.

c) One of the earliest computers built on the Von Neumann Architecture was the EDSAC (Electronic Delay Storage Automatic Calculator), completed at Cambridge in 1949. Other acceptable answers include the Manchester Baby (Small-Scale Experimental Machine, 1948), the first stored-program machine to run a program, or the EDVAC itself.

Marking points:

- *1 mark: names the CPU (Control Unit and/or ALU) as a main component*
- *1 mark: names Main Memory (the store holding both instructions and data) as a main component*
- *1 mark: defines general-purpose computer as programmable/stored-program, able to run different programs to perform many different tasks rather than being fixed to one function*
- *1 mark: names a valid early Von Neumann machine (e.g., EDSAC, Manchester Baby, EDVAC)*
- *1 mark: answer is coherent and uses correct terminology across the subparts (or awarded for a second valid component/justification where the scheme allows)*

Model confidence: high.

Model note: Unofficial model answer, not an official UTM marking scheme. The question asks for TWO components in (a) and ONE machine in (c); the 5-mark allocation is split as 2+2+1 across the subparts, which is an assumption since the per-subpart breakdown is not stated in the extracted text.

QUESTION A2 (5 marks)

Source: CSC429-2025-07, page [2]

Unofficial model answer:

a) The integrated circuit (IC) was first introduced in the third generation of computers (roughly 1964–1971), which replaced discrete transistors with integrated circuits on a single silicon chip.

b) The integrated circuit was invented by Jack Kilby (at Texas Instruments, 1958); Robert Noyce (Fairchild Semiconductor) independently developed a practical silicon IC shortly afterwards, and both are often credited, but the standard textbook answer for the inventor is Jack Kilby.

c) The three main components of an integrated circuit are: (1) the active semiconductor devices — transistors (and diodes) formed on the silicon substrate; (2) passive components — resistors and capacitors; and (3) the interconnection wiring (metal tracks/conductors) together with the silicon substrate/chip on which these elements are fabricated and interconnected.

Marking points:

- *1 mark (a): Third generation (3rd generation) of computers.*
- *1 mark (b): Jack Kilby (accept Robert Noyce as an alternative/co-inventor).*
- *3 marks (c): 1 mark each for any three of: transistors (active devices), resistors, capacitors, diodes, interconnection*

© Hak Cipta Universiti Teknologi MARA

wiring/conductors, silicon substrate/chip.

Model confidence: high.

Model note: No figure or table was supplied; answer is based on standard CSC429 computer-generation material. Some textbooks list the three IC components as transistors, resistors and capacitors, while others include diodes and interconnections — accept any three valid components.

QUESTION A3 (5 marks)

Source: CSC429-2025-07, page [2]

Unofficial model answer:

a) A bit (binary digit) is the smallest unit of information in a computer, representing exactly one of two states, conventionally 0 or 1. Physically it can be realised by any two-state device (a vacuum tube ON/OFF, a transistor, a magnetic cell, a capacitor charge). It is the basic building block from which larger units are formed: 1 nibble = 4 bits, 1 byte = 8 bits, 1 word = typically 16/32/64 bits.

b) Workflow: (1) interpret 'gro' as the decimal number 90 (g=9, r=0 in the usual base-10 reading; this is the only reading that makes the question well-posed). (2) Convert 90 into the required base. (3) Count the digits in that representation, since each digit is stored in one vacuum tube (one tube = one bit-position/state).

i) Binary system: $90 = 64 + 16 + 8 + 2 = 1011010_2$. This representation has 7 binary digits, so 7 vacuum tubes are needed (each tube holds one bit, ON=1/OFF=0). Check: 2^6 = 64 ≤ 90 < 128 = 2^7, so $\lceil \log_2(91) \rceil = 7$ bits.

ii) Octal system: $90 = 1\times64 + 3\times8 + 2 = 132_8$. In an octal system each octal digit is itself represented by 3 bits (a group of 3 tubes), so the number of tubes = 3 digits × 3 bits/digit = 9 tubes. Equivalently 132_8 = 001 011 010_2 = 9 bits. (If the convention is that one tube stores one octal digit directly, the count would be 3 tubes; the standard bit-per-tube interpretation gives 9.)

Marking points:

• *1 mark (a): defines a bit as the smallest two-state unit of information (0/1), correctly identifying it as a binary digit.*

• *1 mark (b): states the workflow — read 'gro' as decimal 90, convert to the target base, then count digits/bits per tube.*

• *1 mark (b)(i): correct binary conversion 90 = 1011010_2.*

• *1 mark (b)(i): concludes 7 vacuum tubes (one tube per bit).*

• *1 mark (b)(ii): correct octal conversion 90 = 132_8 and concludes 9 tubes (3 bits per octal digit), with working shown.*

Model confidence: medium.

Model note: The question text is garbled: '\Mat is a bit?' is read as 'What is a bit?', and 'gro' is assumed to mean the decimal value 90 (digits g=9, r=0). If 'gro' was intended as a different literal or a base-notation value, the tube counts change accordingly. Also, part (b)(ii) is ambiguous: if one tube is taken to store one octal digit, the answer is 3 tubes; the standard one-bit-per-tube interpretation gives 9. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION A4 (5 marks)

Source: CSC429-2014-06, page [5]

Unofficial model answer:

a) The addressing technique illustrated in Figure 2 is **direct addressing** (also called absolute addressing). The instruction contains an Opcode plus an Address field A, and that address A is used directly as the memory address of the operand — the operand is fetched from memory location A with no further calculation.

b) How it is used: The CPU decodes the instruction, takes the address field A from the instruction, and places A directly on the memory address bus (MAR ← A). The memory is then read and the operand stored at location A is transferred to the data bus and into the CPU (MDR/accumulator). The effective address is simply EA = A, so no address arithmetic or index/pointer dereferencing is required; the operand is accessed in one memory reference after instruction fetch.

c) Advantage: It is simple and fast — the effective address is obtained directly from the instruction with no extra address computation, and only one operand memory access is needed (good for accessing fixed/global

© Hak Cipta Universiti Teknologi MARA

variables). Disadvantage: The address field must be large enough to hold a full memory address, so the instruction is long (limited addressable range for a given instruction word length), and it is inflexible — the address is fixed at assembly time, so it cannot easily support relocation, arrays, or pointer/indexed access.

Marking points:

• 1 mark (a): Correctly names direct addressing / absolute addressing.

• 2 marks (b): Explains that address field A is used directly as the operand's memory address (EA = A); 1 mark for stating the operand is fetched from memory location A, 1 mark for noting no address calculation/one memory access after fetch.

• 1 mark (c): One valid advantage (e.g., simple/fast, no address computation, single operand fetch).

• 1 mark (c): One valid disadvantage (e.g., long instruction/large address field, limited address range, inflexible/fixed address, poor for arrays/relocation).

Model confidence: high.

Model note: Figure 2 is supplied only by caption/extracted text; the answer assumes the standard diagram of an instruction with Opcode + Address A pointing to a memory operand, which is the conventional depiction of direct addressing. If the actual figure showed an indirect pointer or index register, the label would differ; this cannot be verified from the caption alone. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION A5 (10 marks)

Source: CSC429-2023-02, page [5]

Unofficial model answer:

a) The three input-output techniques are: (1) Programmed I/O — the CPU directly controls the I/O operation by repeatedly polling the device status register; the CPU is fully occupied during the transfer, so it is simple but inefficient. (2) Interrupt-driven I/O — the CPU issues an I/O command and continues executing other instructions; when the device is ready it raises an interrupt, and the CPU suspends its current program to service the data transfer via an interrupt service routine. This frees the CPU from polling but still requires CPU involvement for every word transferred. (3) Direct Memory Access (DMA) — a DMA controller takes over the bus and transfers blocks of data directly between the device and main memory without CPU intervention; the CPU is interrupted only once per block, making it the most efficient technique for large transfers.

b) A 'hot connection' means that a device can be connected to or disconnected from the USB port while the computer is powered on and running, without switching off the system or rebooting. The USB interface supports dynamic detection and configuration (plug-and-play), so the operating system automatically recognises the new device and loads the appropriate driver, allowing safe insertion and removal at any time.

c) An I/O module is needed as a mediator because: (i) there is a wide variety of peripheral devices with different data formats, speeds and control methods, so the module provides a standard interface to the CPU and hides device-specific details; (ii) devices operate at much slower speeds than the CPU and memory, so the module buffers data and synchronises transfers, preventing the fast CPU from being slowed to device speed; (iii) the module performs control and timing functions, handles interrupts and DMA requests, and relieves the CPU of low-level device management, allowing the CPU to concentrate on computation.

Marking points:

• 1 mark: correctly naming programmed I/O and describing CPU polling/occupation

• 1 mark: correctly naming interrupt-driven I/O and describing interrupt-based transfer

• 1 mark: correctly naming DMA and describing direct memory-to-device block transfer without CPU per-word involvement

• 1 mark: explaining that hot connection allows connecting/disconnecting devices while the system is powered on

• 1 mark: mentioning automatic detection/configuration (plug-and-play) without reboot

• 1 mark: stating device diversity/different formats and need for a standard interface

• 1 mark: stating speed mismatch and need for buffering/synchronisation

• 1 mark: stating that the module handles control/timing and relieves the CPU of device management

• 1 mark: overall clarity and use of correct terminology across all three subparts

© Hak Cipta Universiti Teknologi MARA

• 1 mark: complete coverage of all three subparts (a, b, c) with no missing part

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question text was provided without any figure or table, so no diagram-based interpretation was required. Mark allocation is indicative and assumes a 10-mark total distributed across the three subparts.

QUESTION A6 (10 marks)

Source: CSC429-2023-02, page [3]

Unofficial model answer:

a) SRAM vs DRAM:

i) Bit storage: SRAM stores each bit in a bistable flip-flop (typically a 4- or 6-transistor latch) that holds its state as long as power is applied; no capacitor is involved. DRAM stores each bit as charge on a tiny capacitor (one transistor plus one capacitor per cell), so the bit is represented by the presence or absence of charge rather than by an active latch.

ii) Speed: SRAM is much faster (access times on the order of a few nanoseconds, no refresh overhead, no sense-and-restore cycle). DRAM is slower (tens of nanoseconds) because reading is destructive and requires sense amplifiers plus a write-back/restore, and because of refresh cycles.

iii) Location/use: SRAM is used for cache memory (L1/L2/L3) and registers/small fast buffers close to the CPU. DRAM is used for main memory (the large, cheaper working store), e.g. DDR SDRAM modules.

b) DRAM needs refreshing because each bit is stored as charge on a capacitor that leaks away through the transistor and parasitic paths. Without periodic restoration the charge would decay and the data would be lost. Therefore the memory controller must periodically read and rewrite (refresh) every row within the retention-time window (typically every few milliseconds), which is why DRAM is volatile and has refresh overhead.

c) SRAM would be placed higher in the memory hierarchy (closer to the CPU). Justification: the hierarchy is ordered by speed and cost per bit, with the fastest, most expensive, smallest memories at the top. SRAM is faster, has no refresh overhead, and consumes less dynamic power per access, but it is larger per bit (more transistors) and hence more expensive, so it is used in small caches near the CPU. DRAM is slower and cheaper per bit, so it sits lower, as main memory.

Marking points:

• 1 mark: SRAM uses a flip-flop/latch (4-6 transistors) to hold a bit; DRAM uses a capacitor (1T1C) storing charge.

• 1 mark: SRAM is faster (no refresh, no restore); DRAM is slower (sense/restore and refresh overhead).

• 1 mark: SRAM used in cache/registers near CPU; DRAM used as main memory.

• 1 mark: DRAM bit is charge on a capacitor that leaks.

• 1 mark: refresh periodically reads and rewrites each row to restore charge before it decays.

• 1 mark: correct statement that DRAM is volatile / refresh overhead exists.

• 1 mark: SRAM placed higher in the hierarchy (closer to CPU).

• 1 mark: justification by speed (SRAM faster).

• 1 mark: justification by cost per bit / density (SRAM more expensive, less dense, so small caches).

• 1 mark: coherent overall comparison linking hierarchy ordering to speed and cost.

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure/table was supplied, so the answer relies only on the question text. Mark allocation is indicative and assumes roughly 3 marks for (a), 3 for (b) and 4 for (c).

QUESTION A7 (10 marks)

Source: CSC429-2015-06, page [5]

Unofficial model answer:

a) Four classifications of multiple-processor architecture: (1) Symmetric Multiprocessor (SMP) — all processors share one main memory and have equal access/status, typically connected via a shared bus; (2)

© Hak Cipta Universiti Teknologi MARA

Non-Uniform Memory Access (NUMA) — multiple processors/nodes each with local memory, where access time to local memory is faster than to remote memory; (3) Cluster — a set of independent computers (nodes) connected by a network working together as a single system; (4) Massively Parallel Processor (MPP) / distributed-memory multicomputer — many processors each with private memory communicating by message passing. (Other acceptable classifications: vector/SIMD processors, multicore, UMA vs NUMA vs COMA memory models, Flynn's SISD/SIMD/MISD/MIMD.)

b) X = Shared-Memory / tightly-coupled multiple-processor architecture (i.e., multiple processors that share a common memory address space). SMP and NUMA are shared-memory (tightly coupled) systems; Cluster is loosely coupled, so the intended grouping is the shared-memory multiprocessor family. (If the lecturer's taxonomy groups all three, X = multiple-processor / parallel architecture; the shared-memory interpretation is the standard one.)

c) Differentiation: SMP has all processors sharing a single physical memory and I/O, connected by a common bus, with uniform memory access time and one operating system image; it is tightly coupled and scales poorly beyond a modest number of processors due to bus/memory contention. A Cluster consists of separate, autonomous computers (each with its own CPU, memory and OS) linked by a high-speed network; it is loosely coupled, has no shared global memory (communication is by message passing), and scales well by adding nodes. Opinion/justification: SMP is easier to manage and control because there is a single OS image, one memory space, uniform access, simpler programming model and centralized administration; a cluster requires distributed OS/middleware, message-passing software, and per-node management, making it harder to administer and program despite better scalability and fault isolation.

Marking points:

• 1 mark: SMP correctly listed/defined (shared single memory, equal processors)
• 1 mark: NUMA correctly listed/defined (local vs remote memory access times differ)
• 1 mark: Cluster correctly listed/defined (independent nodes over a network)
• 1 mark: fourth valid classification (e.g., MPP/distributed-memory, vector/SIMD, multicore, Flynn MIMD)
• 1 mark: X correctly identified as shared-memory / tightly-coupled multiprocessor architecture
• 1 mark: SMP described as tightly coupled, single shared memory, uniform access, common bus
• 1 mark: Cluster described as loosely coupled, independent nodes, message passing, no shared global memory
• 1 mark: clear contrast (coupling, memory model, scalability, communication)
• 1 mark: justified opinion on which is easier to manage (single OS image / uniform memory / simpler programming for SMP)
• 1 mark: coherent justification tied to management and control (administration, programming model, scalability trade-off)

Model confidence: medium.

Model note: Part (b) is ambiguous: the paper's 'X' likely refers to the shared-memory/tightly-coupled family (SMP+NUMA), but Cluster is loosely coupled, so if the lecturer's taxonomy intends all three, X = multiple-processor/parallel architecture. Answer states the standard interpretation and flags the alternative. No figure supplied.

QUESTION A8 (10 marks)

Source: CSC429-2025-02, page [3]

Unofficial model answer:

Part (a) — Concept identification (4 marks). The four scenarios map onto the classic *architecture vs. organization* distinction. (i) "Deals with the logical aspects of the system, including instruction sets, data formats, and addressing modes" → **Computer Architecture** (the programmer-visible, logical view). (ii) "Defines the functionality, capabilities, and programming model" → **Computer Architecture** (it specifies *what* the machine can do and how it is programmed). (iii) "Concerned with how the computer will execute the tasks and implement the functions" → **Computer Organization** (it specifies *how* the architectural functions are realised). (iv) "Involves the hardware layout, circuits, control signals, memory access, and timing" → **Computer Organization** (the physical/implementation level). In short: architecture = *what* the system does

© Hak Cipta Universiti Teknologi MARA

(logical, programmer-visible); organization = *how* it does it (physical, implementation-dependent).

Part (b) — Three CPU registers used in data processing (6 marks). (1) **Accumulator (AC)** — a general-purpose data register that holds one operand before an arithmetic/logic operation and receives the result (e.g., ADD loads the operand and stores the sum), so it is central to ALU data processing. (2) **General-purpose registers (GPRs)** — a small set of fast registers (e.g., R0–R15) that hold operands, intermediate results and data being manipulated by arithmetic/logic instructions, reducing memory traffic. (3) **Status/Flag register (Program Status Word, PSW)** — records the outcome/condition of the last arithmetic operation (zero, carry, sign, overflow flags) which is used for conditional branching and to reflect the result of processing. (Acceptable alternatives that also participate in data processing: **Data Register (DR/MDR)** buffering data to/from the ALU and memory, and **Temporary registers** inside the ALU holding operands during an operation.) Each register should be named and its role in arithmetic/data processing explained.

Marking points:

• Part (a): 1 mark each for the four rows — (i) Computer Architecture, (ii) Computer Architecture, (iii) Computer Organization, (iv) Computer Organization (4 marks total).

• Part (b): 1 mark for correctly naming each of three registers (3 marks).

• Part (b): 1 mark for a correct description of each register's role in data/arithmetic processing (3 marks).

• Part (b) total 6 marks; award partial credit where a register is named but its function is vague or incorrect.

• Overall: 4 + 6 = 10 marks.

Model confidence: high.

Model note: The scenario table is supplied only as extracted text (no figure); the mapping to architecture/organization is inferred from standard CSC429 definitions (architecture = logical/programmer-visible; organization = physical/implementation). Part (b) accepts any three valid data-processing registers; the model answer lists the most common ones plus acceptable alternatives.

QUESTION A9 (10 marks)

Source: CSC429-2018-01, page [3]

Unofficial model answer:

a) A complete machine cycle consists of three phases: Fetch, Decode, and Execute. The exact location of each phase is within the CPU and memory: Fetch occurs in the CPU (control unit and program counter) and memory (instruction fetch); Decode occurs in the CPU (instruction decoder/control unit); Execute occurs in the CPU (ALU, registers) and may involve memory or I/O. A typical diagram shows the cycle as a loop: Fetch → Decode → Execute → (repeat). The phases are located as follows: Fetch: PC → MAR, memory read, MBR → IR, PC increment; Decode: IR → control unit, generate control signals; Execute: perform operation using ALU/registers, possibly memory read/write or I/O. (Diagram should clearly label each phase and its location.)

b) Fetch: The CPU fetches the next instruction from memory. The program counter (PC) holds the address of the next instruction; this address is copied to the memory address register (MAR). A memory read signal is issued, and the instruction is placed in the memory buffer register (MBR) or memory data register (MDR), then copied to the instruction register (IR). The PC is incremented to point to the next instruction. Decode: The instruction in the IR is decoded by the control unit to determine the operation to be performed and the operands involved. The control unit generates the necessary control signals for the execution phase. Execute: The CPU performs the operation specified by the instruction. This may involve the arithmetic logic unit (ALU) for arithmetic/logic operations, reading or writing data from/to registers or memory, or performing I/O. The result is stored in a register or memory, and status flags may be updated.

Marking points:

• 1 mark: Correct diagram showing Fetch, Decode, Execute phases in sequence (loop).

• 1 mark: Correct location of Fetch phase (CPU and memory, PC, MAR, MBR, IR).

• 1 mark: Correct location of Decode phase (CPU control unit/decoder).

• 1 mark: Correct location of Execute phase (CPU ALU/registers, possibly memory/I/O).

• 1 mark: Clear description of Fetch phase (PC→MAR, memory read, MBR→IR, PC increment).

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Clear description of Decode phase (instruction decoded, control signals generated).

• 1 mark: Clear description of Execute phase (operation performed, ALU/memory/I/O).

• 1 mark: Mention of PC increment during fetch.

• 1 mark: Mention of control unit role in decode.

• 1 mark: Mention of ALU/registers in execute.

Model confidence: high.

Model note: The question asks for a diagram; the answer describes the expected diagram. No figure is provided, so the diagram is based on standard machine cycle. Marking scheme is unofficial and based on typical expectations.

QUESTION A10 (10 marks)

Source: CSC429-2013-12, page [2]

Unofficial model answer:

The table is best completed by matching each generation to its characteristic technology and a representative machine/inventor. (i) Zero generation technology: mechanical (already given); the machine listed is the Difference Engine (Charles Babbage), and the ABC (Atanasoff–Berry Computer) is also a zero-generation electronic-mechanical special-purpose machine. (ii) The inventor paired with the ABC is John V. Atanasoff and Clifford Berry (already given). (iii) Gottfried von Leibniz is associated with the stepped Reckoner (Leibniz wheel), a mechanical calculator of the zero/pre-generation era. (iv) First-generation technology: vacuum tubes (thermionic valves). (v) First-generation machine: ENIAC (Electronic Numerical Integrator and Computer), by John W. Mauchly and J. Presper Eckert (the table's 'EMAC' is a typo for ENIAC). (vi) The machine by Maurice Wilkes is EDSAC (Electronic Delay Storage Automatic Calculator), the first stored-program computer using mercury delay-line memory. (vii) Second-generation technology: transistors. (viii) Third-generation machine: IBM System/360 family (representative of integrated-circuit mainframes); Jack Kilby is credited with the integrated circuit. (ix) Fourth-generation technology: microprocessors / VLSI (large-scale integration). (x) Fifth-generation technology: artificial intelligence / parallel processing (ULSI, knowledge-based systems).

Marking points:

• 1 mark: (i) Difference Engine / mechanical calculator (Babbage) — zero generation machine

• 1 mark: (ii) Atanasoff–Berry Computer (ABC) — already stated; accept John V. Atanasoff and Clifford Berry

• 1 mark: (iii) Leibniz stepped Reckoner / mechanical calculator (Leibniz wheel)

• 1 mark: (iv) Vacuum tubes (thermionic valves) — first-generation technology

• 1 mark: (v) ENIAC — first-generation machine (Mauchly and Eckert)

• 1 mark: (vi) EDSAC — Wilkes' stored-program machine

• 1 mark: (vii) Transistors — second-generation technology

• 1 mark: (viii) IBM System/360 (or representative IC mainframe) — third-generation machine

• 1 mark: (ix) Microprocessors / VLSI — fourth-generation technology

• 1 mark: (x) Artificial intelligence / parallel processing (ULSI) — fifth-generation technology

Model confidence: medium.

Model note: The extracted table text is garbled ('Ceperatlon', 'Anatasoft', 'EMAC'); ENIAC is assumed for 'EMAC'. Some cells (e.g., zero-generation machine, third-generation machine) are ambiguous because the table mixes technology, machine and inventor columns; answers reflect standard CSC429 generation classifications. This is an unofficial model answer, not an official UTM marking scheme.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Source: CSC429-2013-12, page [6]

Unofficial model answer:

Subpart (a) — One-address instructions. In a one-address machine every arithmetic instruction implicitly uses an accumulator (here Register X), and the single operand is a memory location. The equation in Figure 4 is taken to be X = (A + B) × (C − D) ÷ (E + F); the figure itself was supplied only as a cropped image, so this expression is an assumption. Program: `LOAD A` (X ← A); `ADD B` (X ← X + B = A + B); `STORE T1` (save A + B); `LOAD C` (X ← C); `SUB D` (X ← C − D); `STORE T2` (save C − D); `LOAD T1` (X ← A + B); `MUL T2` (X ← (A+B)(C−D)); `STORE T3`; `LOAD E` (X ← E); `ADD F` (X ← E + F); `STORE T4`; `LOAD T3` (X ← numerator); `DIV T4` (X ← (A+B)(C−D)/(E+F)); `STORE X` (final answer in X). Temporary memory locations T1–T4 are needed because only one operand can be addressed per instruction.

Subpart (b) — Two-address instructions. A two-address instruction has the form OP (X ← Y op Z) or OP (X ← X op Y), destroying one source. Using the register-to-register forms from Table 2: `MOVE X, A` (X ← A); `ADD X, B` (X ← A + B); `MOVE Y, C` (Y ← C); `SUB Y, D` (Y ← C − D); `MUL X, Y` (X ← (A+B)(C−D)); `MOVE Z, E` (Z ← E); `ADD Z, F` (Z ← E + F); `DIV X, Z` (X ← (A+B)(C−D)/(E+F)). The final result is already in X, so no STORE is required. Only the temporary registers X, Y and Z are used; A–F are never overwritten.

Subpart (c) — Three-address instructions. A three-address instruction specifies two sources and a distinct destination, OP (X ← Y op Z), so no intermediate results need to be saved and no source is destroyed. Program: `ADD X, A, B` (X ← A + B); `SUB Y, C, D` (Y ← C − D); `MUL X, X, Y` (X ← (A+B)(C−D)); `ADD Z, E, F` (Z ← E + F); `DIV X, X, Z` (X ← (A+B)(C−D)/(E+F)). The answer ends in X as required. This is the shortest program, illustrating that three-address format minimises instruction count at the cost of a longer instruction encoding.

Marking points:

- *2 marks: correct identification/statement of the expression to be computed from Figure 4 (e.g. X = (A+B)(C−D)/(E+F))*
- *6 marks: one-address program — correct use of LOAD/STORE/ADD/SUB/MUL/DIV with accumulator X, correct temporary storage of partial results, correct order of operations, final result in X (deduct for missing STORE or wrong operand order)*
- *5 marks: two-address program — correct MOVE/ADD/SUB/MUL/DIV (X←Y op Z or X←X op Y) forms, correct use of temporary registers X, Y, Z, A–F unmodified, final result in X*
- *5 marks: three-address program — correct OP (X←Y op Z) form, minimal instruction sequence, A–F unmodified, final result in X*
- *2 marks: correct assembly notation and explanatory comments throughout; no modification of registers A–F*

Model confidence: low.

Model note: Figure 4 was supplied only as a cropped image with no readable expression; the equation X = (A+B)(C−D)/(E+F) is assumed from typical past-paper style and must be checked against the actual figure. Table 2 text is garbled (e.g. 'MOVE (X e Y)', 'ADD (X k.. X + Y)'), so instruction mnemonics/operand orders are interpreted as standard one-, two- and three-address forms. This is an unofficial model answer, not an official UTM marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA